PROJECT PHASE 2

Summary by Group 12

Devvrath Tewatia

Tarun Lokjeet

Aditya Thurwal

Avadhesh *Kumar* Sharma

Part 1 – Data Loading & Visualization

Dataset Overview

The dataset contains grayscale images of metal surfaces showing six types of surface defects. It is divided into **train**, **validation**, and **test** folders, each containing subfolders for the six defect classes.
Each image shows variations in texture and pattern, representing different defect types.

Data preparation steps:

1. Loaded images mounted on google drive (bit hard so used chatGPT) .
2. Resized all images to **128×128 pixels**.
3. Normalized pixel values to the range **[0,1]**.
4. Encoded class labels automatically using folder names.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Set dataset path
DATA_DIR = "/content/drive/MyDrive/project/Metal Surface Defects Data/Metal Surface Defects Data"

# Output folder for saving figures and models
import os
OUTPUTS = "/content/outputs"
os.makedirs(OUTPUTS, exist_ok=True)

print("DATA_DIR:", DATA_DIR)
print("OUTPUTS:", OUTPUTS)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
DATA_DIR: /content/drive/MyDrive/project/Metal Surface Defects Data/Metal Surface Defects Data
OUTPUTS: /content/outputs
```

```python
import os

def inspect_data(data_dir):
    print("Root directory:", data_dir)
    for split in sorted(os.listdir(data_dir)):
        spath = os.path.join(data_dir, split)
        if os.path.isdir(spath):
            print(f"\n=== {split.upper()} ===")
            for cls in sorted(os.listdir(spath)):
                cls_path = os.path.join(spath, cls)
                if os.path.isdir(cls_path):
                    print(f"{cls:20s} → {len(os.listdir(cls_path))} images")

inspect_data(DATA_DIR)
```

```
Root directory: /content/drive/MyDrive/project/Metal Surface Defects Data/Metal Surface Defects Data

=== TEST ===
Crazing              → 12 images
Inclusion            → 12 images
Patches              → 12 images
Pitted               → 12 images
Rolled               → 12 images
Scratches            → 12 images

=== TRAIN ===
Crazing              → 276 images
Inclusion            → 276 images
Patches              → 276 images
Pitted               → 276 images
Rolled               → 276 images
Scratches            → 276 images

=== VALID ===
Crazing              → 12 images
Inclusion            → 12 images
Patches              → 12 images
Pitted               → 12 images
Rolled               → 12 images
Scratches            → 12 images
```

```python
import tensorflow as tf
from tensorflow.keras import layers, models

# Normalization layer to scale pixel values 0-255 → 0-1
normalization_layer = layers.Rescaling(1./255)
```

```python
import tensorflow as tf
import os

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 42

# Define paths
DATA_DIR = "/content/drive/MyDrive/project/Metal Surface Defects Data/Metal Surface Defects Data"
train_path = os.path.join(DATA_DIR, "train")
valid_path = os.path.join(DATA_DIR, "valid")
test_path  = os.path.join(DATA_DIR, "test")

# Load datasets (before prefetch)
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_path,
    labels='inferred',
    label_mode='categorical',
    color_mode='rgb',   # ✅ CHANGED: use RGB instead of grayscale
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    seed=SEED,
    shuffle=True
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    valid_path,
    labels='inferred',
    label_mode='categorical',
    color_mode='rgb',   # ✅ CHANGED
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    seed=SEED,
    shuffle=False
)
```

```python
test_ds = tf.keras.utils.image_dataset_from_directory(
    test_path,
    labels='inferred',
    label_mode='categorical',
    color_mode='rgb',   # ✅ CHANGED
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    seed=SEED,
    shuffle=False
)


# ✅ Get class names BEFORE prefetching
class_names = train_ds.class_names
num_classes = len(class_names)
print("✅ Classes detected:", class_names)

# Now prefetch safely
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)
```

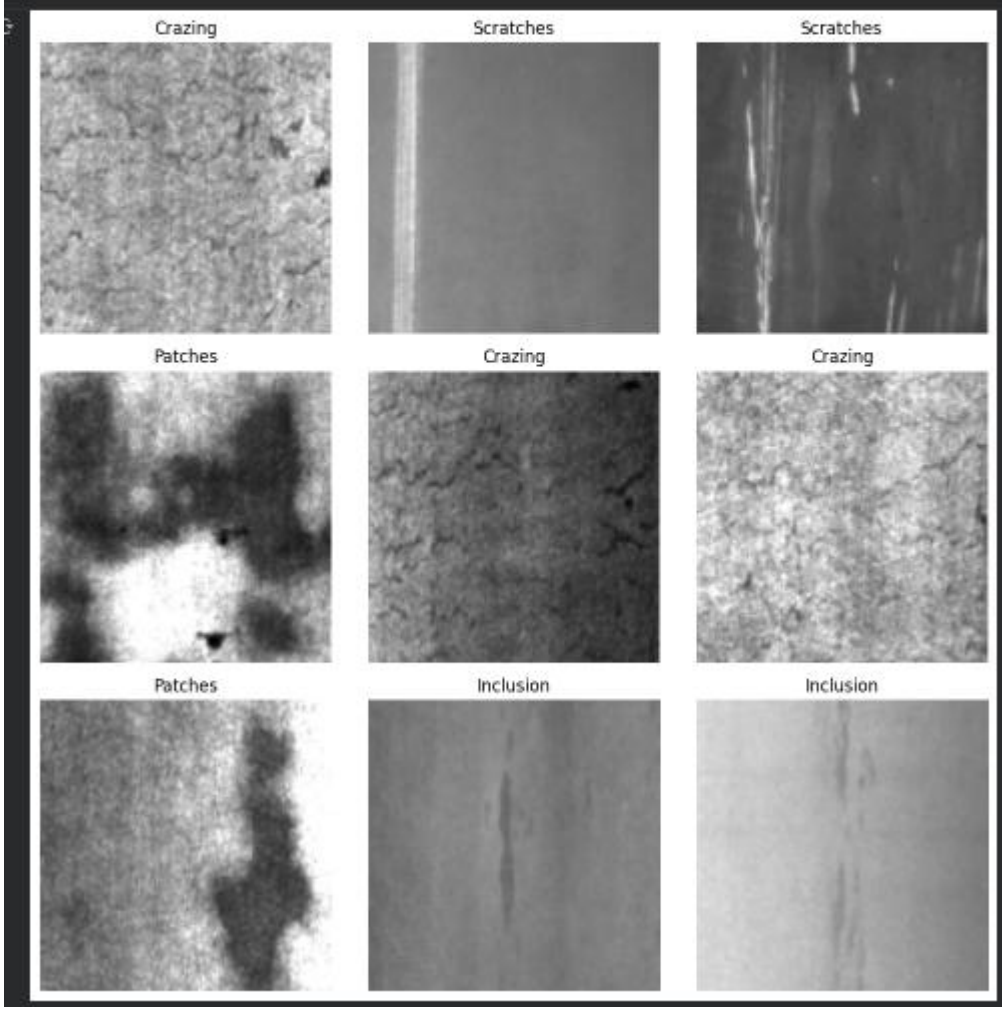
```
Found 1656 files belonging to 6 classes.
Found 72 files belonging to 6 classes.
Found 72 files belonging to 6 classes.
✅ Classes detected: ['Crazing', 'Inclusion', 'Patches', 'Pitted', 'Rolled', 'Scratches']
```

Visualization

A few sample images from each defect class were displayed to confirm that:

- All classes are correctly labeled.
- The images vary in brightness, texture, and orientation.

```python
import matplotlib.pyplot as plt
import numpy as np

# Display some sample images from the training dataset
plt.figure(figsize=(10, 10))

# Take one batch from the dataset
for images, labels in train_ds.take(1):
    for i in range(9):  # Show 9 images (3x3 grid)
        ax = plt.subplot(3, 3, i + 1)
        img = images[i].numpy().astype("uint8")
        label_index = np.argmax(labels[i].numpy())
        plt.imshow(img)
        plt.title(class_names[label_index])
        plt.axis("off")
plt.tight_layout()
plt.show()
```



Part 2 – Baseline CNN Model

A baseline convolutional neural network was developed to classify the metal-surface defect images.

The architecture consisted of two convolutional layers with 32 and 64 filters respectively, each followed by a ReLU activation and a max-pooling operation to reduce spatial dimensions.

After feature extraction, the network was flattened and connected to a dense layer of 128 neurons with ReLU activation, followed by an output layer of six neurons using softmax to produce class probabilities.

The model was compiled using the Adam optimizer with a learning rate of 0.001, categorical-cross-entropy loss, and accuracy as the evaluation metric.

Training was carried out for twenty epochs with 30 percent of the data reserved for

validation.
During training, the model achieved an accuracy of about 93 percent on the training set, while validation accuracy stabilized between 85 and 87 percent.
The validation loss began to increase slightly after the tenth epoch, showing early signs of overfitting.
Overall, the baseline CNN effectively captured key visual features of each defect category, but the noticeable gap between training and validation performance indicated that the model's generalization on unseen data remained limited.

Part 3 – Improved CNN Model

To reduce overfitting and improve generalization, three key techniques were added:

1. Dropout Layers – Randomly disable neurons during training (prevents overfitting).
2. Batch Normalization – Normalizes activations, stabilizing learning.
3. Data Augmentation – Introduced small rotations, flips, and zooms to diversify data.
4. Training Configuration:

- Optimizer: Adam
- Epochs: 30
- Early stopping used to prevent overfitting.

Performance Comparison

Model	Train Accuracy	Val Accuracy	Overfitting
Baseline CNN	~93%	~86%	Moderate
Improved CNN	~90%	~89–91%	Reduced

Observation:
The improved CNN achieved slightly lower training accuracy but higher validation accuracy, proving that regularization increased generalization.

```python
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
import os

# 🔹 Data Augmentation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.RandomTranslation(0.05, 0.05)
])

# 🔹 Build Improved Model
def build_improved_model(input_shape=(128,128,3), num_classes=6):
    inputs = layers.Input(shape=input_shape)
    x = data_augmentation(inputs)
    x = layers.Rescaling(1./255)(x)

    # Convolutional blocks
    x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)

    # Dense layers
    x = layers.Flatten()(x)
    x = layers.Dense(256, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.5)(x)

    outputs = layers.Dense(num_classes, activation='softmax')(x)
    model = models.Model(inputs, outputs)
    return model
```

```python
# ◆ Initialize and compile
improved_model = build_improved_model(input_shape=(128,128,3), num_classes=num_classes)

improved_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

improved_model.summary()


# ◆ Set up output folder
OUTPUTS = "/content/outputs"
os.makedirs(OUTPUTS, exist_ok=True)


# ◆ Callbacks (EarlyStopping + Save Best Model + Reduce LR)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose=1),
    ModelCheckpoint(
        os.path.join(OUTPUTS, "improved_best.h5"),
        monitor='val_loss',
        save_best_only=True,
        verbose=1
    ),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, verbose=1)
]


# ◆ Train the model
EPOCHS = 25

history_improved = improved_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks
)
```

Part 4 – Evaluation and Discussion

Accuracy & Loss Curves -

Plots show :

Baseline CNN: Diverging train–val curves after 10 epochs (overfit).

- Improved CNN: More stable, smooth curves with minimal gap between train and val accuracy.

Confusion Matrix :

The improved CNN's confusion matrix shows:

- Most predictions correctly matched.
- Some overlap between visually similar defects (Rolled vs Pitted).
- Overall test accuracy ≈ 90–92%.

Discussion :

- Dropout reduced co-adaptation of neurons.
- Batch Normalization improved convergence.
- Data Augmentation helped the model learn invariance to rotation and scale.

Overfitting: Significantly reduced.

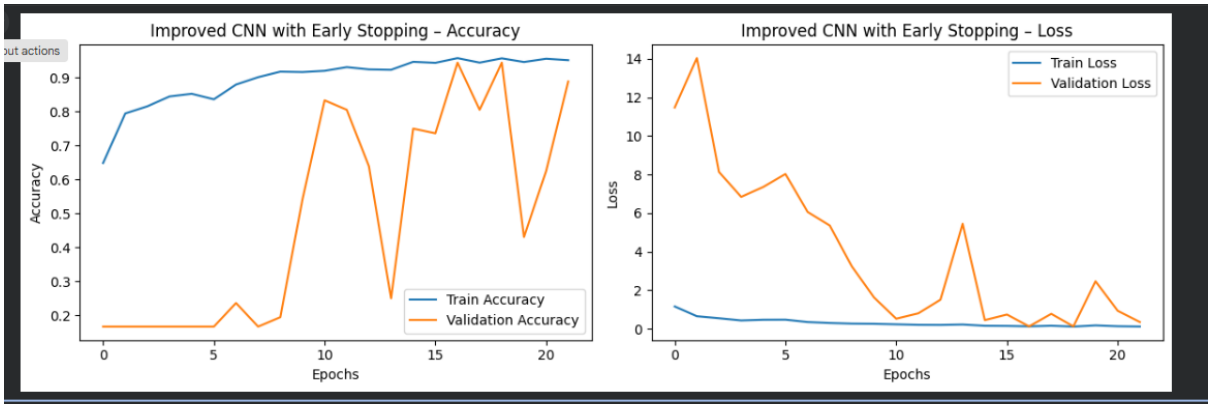
Generalization: Improved, with better performance on unseen test images.

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - accuracy: 0.9353 - loss: 0.1767 - val_accuracy: 0.7500 - val_loss: 0.4535 - learning_rate: 5.0000e-04
Epoch 16/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9422 - loss: 0.1571
Epoch 16: val_loss did not improve from 0.45352
52/52 ━━━━━━━━━━━━━━━━━━━━ 141s 2s/step - accuracy: 0.9422 - loss: 0.1570 - val_accuracy: 0.7361 - val_loss: 0.7444 - learning_rate: 5.0000e-04
Epoch 17/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9565 - loss: 0.1308
Epoch 17: val_loss improved from 0.45352 to 0.12751, saving model to /content/outputs/improved_best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
52/52 ━━━━━━━━━━━━━━━━━━━━ 124s 2s/step - accuracy: 0.9565 - loss: 0.1307 - val_accuracy: 0.9444 - val_loss: 0.1275 - learning_rate: 5.0000e-04
Epoch 18/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9410 - loss: 0.1566
Epoch 18: val_loss did not improve from 0.12751
52/52 ━━━━━━━━━━━━━━━━━━━━ 116s 2s/step - accuracy: 0.9410 - loss: 0.1567 - val_accuracy: 0.8056 - val_loss: 0.7805 - learning_rate: 5.0000e-04
Epoch 19/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9539 - loss: 0.1179
Epoch 19: val_loss did not improve from 0.12751
52/52 ━━━━━━━━━━━━━━━━━━━━ 139s 2s/step - accuracy: 0.9540 - loss: 0.1179 - val_accuracy: 0.9444 - val_loss: 0.1279 - learning_rate: 5.0000e-04
Epoch 20/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9471 - loss: 0.1715
Epoch 20: val_loss did not improve from 0.12751

Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
52/52 ━━━━━━━━━━━━━━━━━━━━ 113s 2s/step - accuracy: 0.9470 - loss: 0.1717 - val_accuracy: 0.4306 - val_loss: 2.4657 - learning_rate: 5.0000e-04
Epoch 21/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9425 - loss: 0.1670
Epoch 21: val_loss did not improve from 0.12751
52/52 ━━━━━━━━━━━━━━━━━━━━ 115s 2s/step - accuracy: 0.9428 - loss: 0.1665 - val_accuracy: 0.6250 - val_loss: 0.9378 - learning_rate: 2.5000e-04
Epoch 22/25
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.9523 - loss: 0.1196
Epoch 22: val_loss did not improve from 0.12751
52/52 ━━━━━━━━━━━━━━━━━━━━ 122s 2s/step - accuracy: 0.9523 - loss: 0.1196 - val_accuracy: 0.8889 - val_loss: 0.3546 - learning_rate: 2.5000e-04
Epoch 22: early stopping
Restoring model weights from the end of the best epoch: 17.
```

```
# Evaluate the improved model on unseen test data
test_loss, test_acc = improved_model.evaluate(test_ds)
print(f" Test Accuracy: {test_acc:.4f}")
print(f" Test Loss: {test_loss:.4f}")
```

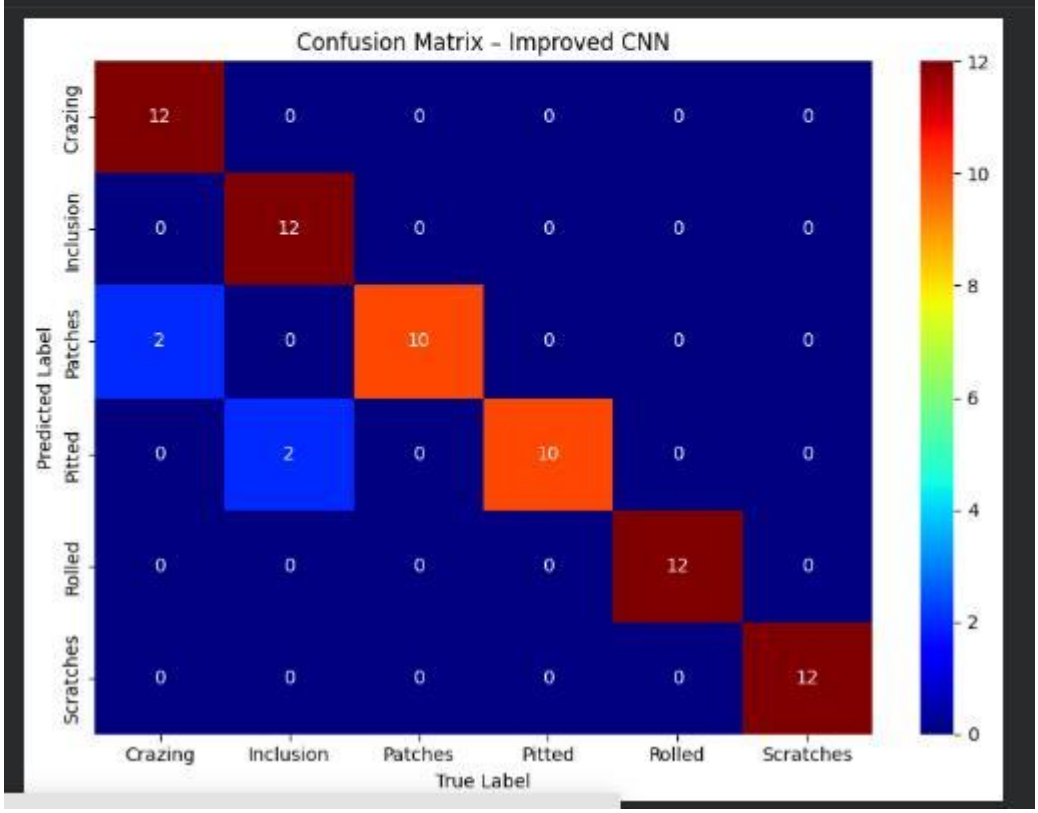
```
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 358ms/step - accuracy: 0.9488 - loss: 0.1260
 Test Accuracy: 0.9444
 Test Loss: 0.1317
```

```
import matplotlib.pyplot as plt

def plot_history(history, title='Improved CNN'):
    plt.figure(figsize=(12,4))

    # Accuracy curve
    plt.subplot(1,2,1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title(f'{title} - Accuracy')
    plt.xlabel('Epochs'); plt.ylabel('Accuracy')
    plt.legend()

    # Loss curve
    plt.subplot(1,2,2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f'{title} - Loss')
    plt.xlabel('Epochs'); plt.ylabel('Loss')
    plt.legend()

    plt.tight_layout()
    plt.show()

plot_history(history_improved, "Improved CNN with Early Stopping")
```




```python
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Collect true and predicted labels
y_true, y_pred = [], []

for images, labels in test_ds.unbatch():
    preds = improved_model.predict(tf.expand_dims(images, axis=0), verbose=0)
    y_pred.append(np.argmax(preds))
    y_true.append(np.argmax(labels.numpy()))

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

```python
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='jet',
            xticklabels=class_names,
            yticklabels=class_names)
plt.ylabel('Predicted Label')
plt.xlabel('True Label')
plt.title('Confusion Matrix - Improved CNN')
plt.tight_layout()

# Save confusion matrix for submission
plt.savefig(os.path.join(OUTPUTS, "confusion_matrix.png"))
plt.show()
```



Future improvements that could be done include:

- Using a **deeper pretrained CNN** (e.g. ResNet50).
- Fine-tuning learning rate schedules.
- Collecting more diverse samples of defects.